CHAPTER 25

S3-like Object Storage

Durable (11 nines), scalable blob storage with buckets, objects, versioning and a metadata service.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design an object storage service like Amazon S3: users create **buckets**, **upload/download objects** (immutable blobs with metadata), list objects by prefix, enable versioning, and rely on very high durability and availability at exabyte scale.

Why interviewers ask it: Storage-systems depth: separating data and metadata stores, replication vs erasure coding, durability math, multipart upload.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Features?	Bucket creation, object upload/download, versioning, list objects in a bucket.
Object size?	Both massive (GBs) and small (KBs); mostly large.
Data volume?	100 PB per year.
Durability / availability?	Durability 6 nines+ (aim 11 nines), availability 4 nines.

3. Requirements

Functional

- Create/delete buckets.
- PUT/GET/DELETE objects (immutable; updates create a new version).
- Object versioning.
- List objects with prefix.
- Multipart upload for large objects.

Non-functional

- **Durability** 99.999999999% (11 nines).
- High availability.
- Scale to exabytes; cost-efficient.
- Low-latency access for small objects, high throughput for big ones.

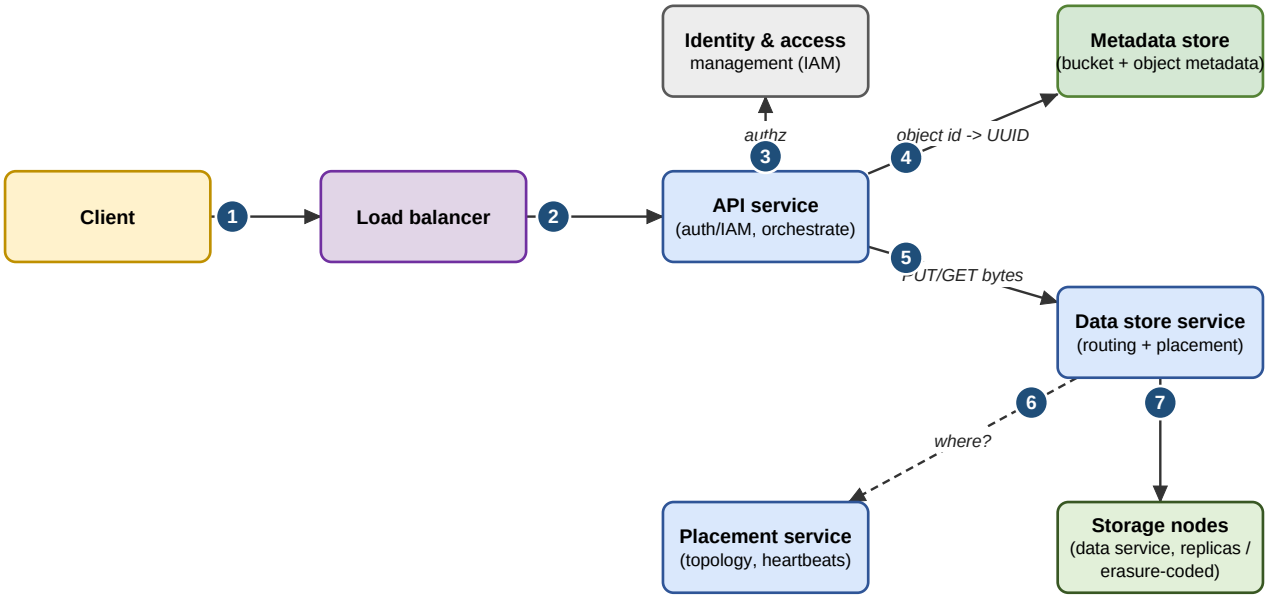
4. Back-of-the-envelope estimation

- Assume object mix: 40% under 1 MB (avg 0.5 MB), 55% between 1 and 64 MB (avg 32 MB), 5% large (avg 200 MB). Weighted average = 0.2 + 17.6 + 10 = about **28 MB per object**.
- 100 PB of new data per year / 28 MB = about **3.6 billion objects per year**. At about 1 KB of metadata each = about **3.6 TB metadata per year** (about 36 TB over 10 years) - tiny compared with the data, but still needs sharding for QPS.
- Raw disk needed per year: 3x replication = 300 PB; erasure coding 8+4 (1.5x) = 150 PB. That 150 PB difference is why EC matters for cost.

5. API / data model

```
PUT /bucket-to-share                      # create bucket
PUT /bucket-to-share/script.txt           # upload object (body = bytes)      GET /bucket-to-share/script.txt?ve
GET /bucket-to-share?prefix=abc&max-keys=100   # list objects (prefix + delimiter)      DELETE /bucket/key (adds
```

6. High-level design



Step	What happens
1	The client sends `PUT /bucket/object` (or a multipart upload) with its credentials through the load balancer.
2	The **API service** terminates the request, validates it and orchestrates the rest. It does not hold the data.
3	It asks **IAM** whether this identity may write to this bucket (policies, ACLs).
4	It allocates an **object ID (UUID)** and records `bucket + name -> object ID` and its attributes in the **metadata store** (sharded by `bucket_id, object_name`).
5	It streams the bytes to the **data store service**, which is immutable (objects are not updated in place; a new version is written).
6	The placement service says which nodes should store the replicas, using topology (rack/AZ) and heartbeats to avoid unhealthy nodes.
7	The data service writes to the **storage nodes**: 3 replicas, or **erasure-coded** shards (for example 8+4) across AZs. It acknowledges only after enough copies are durable, and the object is then visible.

7. Deep dive

Separate data store and metadata store

- Like a UNIX file system: **inode (metadata)** vs **file blocks (data)**. Object = (metadata: name, bucket, version, size, checksum, UUID pointing to the data; data: immutable bytes).
- Metadata DB: bucket table and object table (`bucket_name`, `object_name`, `object_version`, `object_id`) sharded by `hash(bucket_name, object_name)`; listing by prefix needs an ordered index (e.g. per-bucket scan) - sharding by bucket_id helps listing at the cost of hot buckets.

Data store design

- **Data routing service** (stateless API to storage nodes), **placement service** (picks nodes, tracks topology via heartbeats, manages replication groups, virtual cluster map; consensus (Paxos/Raft) for HA), **data nodes** (store objects as **append-only WAL files**).
- Small objects are **packed into large files** (e.g. 1 GB read-write file) to avoid inode exhaustion and many tiny files; an object-id -> (file, offset, size) mapping is held in SQLite/RocksDB per node. When a file is full it becomes read-only.

Upload (PUT) flow

- Client -> API (authN/authZ via IAM) -> data store service writes the object to a primary node which replicates to secondaries -> returns UUID -> metadata record inserted -> 200 OK. Commit only after enough replicas persist.

Download (GET) flow

- Look up metadata (`bucket, object, version`) to get the UUID and checksum -> data service finds the node holding it -> read -> verify checksum -> stream to client.

Durability: replication vs erasure coding

- **3x replication** across AZs/racks: simple, fast, 200% overhead, durability ~ 6-9 nines depending on failure model.
- **Erasure coding (e.g. 8+4):** split into 8 data + 4 parity chunks across nodes; tolerates any 4 losses with 50% overhead (1.5x) and gives ~ 11 nines; costs CPU and slower reads/repair (reconstruct from several nodes). Typically used for large/cold objects; replication for small/hot ones.
- Durability math: node annual failure rate (AFR) ~ 0.81%; with repair windows and spread across failure domains, 11 nines is achieved with EC across 3 AZs.

Correctness: checksums

- Compute a checksum (MD5/SHA/CRC) at ingest and verify on read, repair and during background **scrubbing** (re-read data and compare); detect bit rot; heal by rebuilding from other copies/parities.

Multipart upload

- Large objects are split into parts (e.g. 5 MB-5 GB) uploaded in parallel and retried individually; a final `complete` call concatenates parts logically (metadata assembly); abandoned parts are garbage collected.

Versioning and deletion

- With versioning, each PUT creates a new version id; DELETE adds a **delete marker**. Without versioning, overwrite replaces the pointer. Physical deletion is **lazy**: garbage collector compacts files, reclaims space from deleted objects, and merges small files.

Listing

- `prefix` + `delimiter` emulate folders; implement with ordered index over object names; pagination via continuation token. Listing is one of the hardest operations in sharded metadata.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Durability technique	3x replication / erasure coding	EC for cost-efficient 11 nines; replication for low latency small objects.
Metadata sharding	By hash(object) / by bucket	Hash for even load; bucket-level ordering for efficient listing (or secondary list index).
Small objects	One file per object / packed into large files	Packed log files avoid inode and filesystem limits.
Consistency	Strong read-after-write / eventual	S3 is now strongly consistent; achieve with consensus on metadata.

9. Scaling, reliability, failure modes

- Stateless API/data routing scale horizontally; placement service small and HA.
- Rack and AZ-aware placement to survive correlated failures; throttled rebuild traffic.
- Tiering (hot/cold/archival) and lifecycle policies for cost.

10. Likely follow-up questions

- How do you reach 11 nines of durability?
- How do you deal with hot buckets/objects?
- How would you implement cross-region replication and lifecycle policies?
- How would you do efficient 'list by prefix' at scale?

11. Common mistakes

- Mixing data and metadata in one store.
- Storing each small object as a separate file.
- No checksum / scrubbing story.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: buckets, put and get immutable objects, versioning, multipart upload for large files, listing by prefix. Assume 100 PB of data, objects from KBs to GBs, and 6 nines durability with 4 nines availability. Most objects are small, but most bytes are in large files, so I need to optimise for both."
2. High-level design	10 min	"I separate metadata from data. An API service authenticates through IAM, stores bucket and object metadata in a sharded metadata store, and sends bytes to a data store service. A placement service decides which storage nodes hold the replicas."
3. Deep dive	25 min	"For durability: replication is simple and fast to read, erasure coding like 8+4 gives high durability at about 1.5x storage overhead but costs more CPU and latency to repair, so I'd use EC for cold data and replication for hot. Small objects are packed into large append-only files with an index, to avoid the small-file problem. Multipart upload lets the client upload parts in parallel and complete atomically. Versioning keeps old object IDs, and garbage collection reclaims deleted data. Checksums plus scrubbing catch bit rot."
4. Wrap up	4 min	"Extras: lifecycle policies and tiering, cross-region replication, strong read-after-write consistency, and monitoring repair backlog and disk failure rate."

13. 60-second recap / practice checklist

- Data store (immutable blobs, packed, replicated/erasure-coded) + metadata store (sharded) + API/IAM + placement service.
- Durability via EC/replication across failure domains, checksums, scrubbing, repair.
- Multipart upload, versioning with delete markers, lazy garbage collection.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• AWS: S3 strong consistency
• AWS: S3 storage classes and durability
• Backblaze: Reed-Solomon erasure coding